# HMS-PED: A Hybrid Multi-Stage Phishing Email Detector Adapting the HMS-IDS Framework, with Large Language Model Escalation in a Browser Extension

Sameer Hembram (S24CSEU0453)
*Dept. of Computer Science and Engineering*
*Bennett University*
Greater Noida, India

Ayushmaan Kumar (S24CSEU2784)
*Dept. of Computer Science and Engineering*
*Bennett University*
Greater Noida, India

***Abstract*—Phishing email remains the most common entry point for cyber attacks. Detectors built on large language models (LLMs) can explain their decisions in plain language, but they are slow, costly per message, send email content to a third party, and are rarely evaluated on more than a handful of emails. We adapt the Hybrid Multi-Stage Intrusion Detection System (HMS-IDS), originally proposed for zero-day attacks in industrial IoT networks, to phishing email detection and integrate it into an existing Gmail browser extension that previously relied on an LLM alone. Stage 1 is a stacking ensemble of Naive Bayes, Decision Tree, $k$-Nearest Neighbours and Logistic Regression over Information-Gain-selected text features and hand-crafted phishing indicators. Stage 2 is the HMS-IDS cluster-labelling $k$-means (CL-$k$-means) anomaly layer, which re-examines emails that Stage 1 considers legitimate. Stage 3 escalates uncertain emails to the Gemini LLM and produces a human-readable explanation. On a stratified 70/15/15 split of 41,794 de-duplicated emails from the public Ling and CEAS-2008 corpora, the stacking ensemble reaches 99.54% accuracy (F1 0.9956, false-positive rate 0.47%). However, when the same pipeline is trained on one corpus and tested on the other, F1 falls to 0.3647 (CEAS→Ling), and the CL-$k$-means layer does not recover it. This exposes a generalisation gap that in-distribution accuracy hides, and it motivates the LLM escalation stage. The code, data splits and results are released for reproducibility.**

***Index Terms*—Phishing detection, ensemble learning, stacking, $k$-means, large language models, browser extension, email security**

## I. Introduction

Phishing emails impersonate trusted senders to steal credentials or deliver malware, and they remain a leading initial-access vector. Classical defences combine blocklists with supervised classifiers trained on labelled emails [1], [2]. More recently, LLMs have been used as zero-shot phishing detectors that also explain their verdicts [3], [4].

Our base paper, Pham *et al.* [5], presented an LLM-based browser extension for phishing detection. This project builds on its open-source implementation: a Chrome extension that reads the open Gmail message and uses Google's Gemini model to decide whether it is phishing [6]. Whilst the design is attractive as all results have an explanation, there are 4 problems: each scan makes an API call over the network, the whole email is sent to a third party, the model can be tricked by text hidden in the email [7], and the evaluation script in its public code tests only 20 emails.

Our reference framework, HMS-IDS [8], faced an analogous problem in network intrusion detection. Supervised models detect *known* attacks well but miss *zero-day* ones. HMS-IDS addresses this with two tiers: a stacking ensemble of four simple learners, followed by an unsupervised cluster-labelling $k$-means layer to review traffic that was deemed benign by the stacking ensemble. We have used this framework and extended it by adding an LLM layer.

The contributions of this work are:

1) **HMS-PED**, an adaptation of HMS-IDS to phishing email detection. It uses Information-Gain feature selection, a stacking ensemble of NB, DT, KNN and LR, and the CL-$k$-means anomaly layer with a cluster-purity threshold $\tau$, extended with an LLM escalation stage.
2) A reproducible evaluation on 41,794 emails with a fixed stratified train/validation/test split. All hyper-parameters are chosen on the validation set only.
3) An *unseen-corpus* protocol, the email analogue of zero-day evaluation, which shows that near-perfect in-distribution scores do not transfer across corpora.
4) Integration of the full pipeline into the original browser extension, including a fix for its email-capture defect.

## II. Related Work

### A. Machine-learning phishing and spam detection

Fette *et al.* [1] showed that a small set of structural features (e.g., number of links, IP-based URLs, HTML content) lets a classifier detect phishing with high accuracy. Abu-Nimeh *et al.* [2] compared logistic regression, trees, random forests, SVMs and neural networks on the same task and found trade-offs between false positives and false negatives across the models. Androutsopoulos *et al.* [9] introduced the Ling-Spam corpus used here and evaluated Naive Bayes filtering on it. Champa

*et al.* [10], [11] curated several public corpora, including Ling and CEAS-2008, into a common format, which we use directly.

### B. LLM-based detection

ChatSpamDetector [3] prompts GPT-family models to classify emails and justify the decision. Heiding *et al.* [4] studied LLMs both for writing and for detecting phishing emails and reported LLM-based detection as promising. LLM detectors also inherit the risks of LLM-integrated applications, notably indirect prompt injection [7]. Pham *et al.* [5] demonstrated an LLM-based browser extension that classifies the open email and explains its verdict; it is the base paper of this work. Beyond email, Rahman *et al.* [12] examine LLMs for the cybersecurity of power distribution grids, a setting close to the industrial-IoT domain of our reference framework, where reliability of and trust in model output are similar concerns to ours.

### C. The reference framework: HMS-IDS

Saurabh *et al.* [8] proposed HMS-IDS for industrial IoT networks. After pre-processing and Information-Gain feature selection, a stacking ensemble of Decision Tree, KNN, Logistic Regression and Naive Bayes, with the best base learner as meta-learner, detects known attacks. Samples the ensemble labels *normal* go to a CL-$k$-means model: clusters are labelled by majority class, and a test sample inherits the label of its nearest cluster together with a confidence equal to the cluster's purity. HMS-IDS reports 99.49% accuracy on CIC-ToN-IoT. This work maintains this two-level hierarchy. However, we change the domain of data used (email text instead of network flows), how features are extracted, and what is considered as 'unknown' threats.

## III. Datasets

We use the Ling and CEAS-2008 corpora as distributed by Champa *et al.* [11], the same files shipped with the original extension [6]. Label 1 denotes a phishing/malicious email and 0 a legitimate one. Subjects and bodies are whitespace-normalised, bodies are truncated to 5,000 characters, empty emails are removed, and exact duplicates of (subject, body) are dropped. This leaves 41,794 emails.

The data are split 70/15/15 into training, validation and test sets, stratified jointly on label and source corpus, with a fixed random seed (Table I). The validation split is used for every tuning decision, and the test split is used once for the final numbers. This differs from HMS-IDS, which reports 10-fold cross-validation on a 70/30 split; we also report 5-fold cross-validation accuracy on the training split for comparison.

TABLE I
Data splits (stratified on label and corpus)

| Split | Emails | Phishing | Legitimate |
|---|---|---|---|
| Train | 29,255 | 15,491 | 13,764 |
| Validation | 6,269 | 3,320 | 2,949 |
| Test | 6,270 | 3,320 | 2,950 |
| Total | 41,794 | 22,131 | 19,663 |

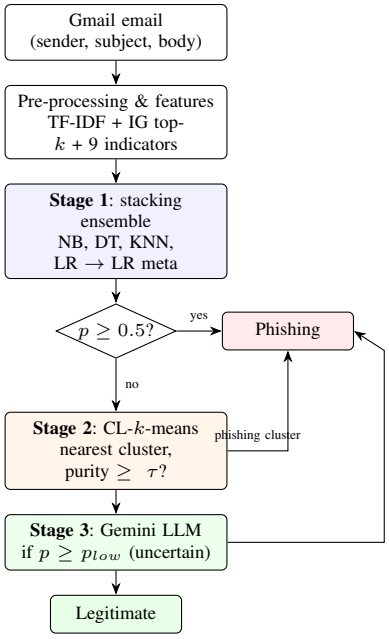



Fig. 1. HMS-PED pipeline. Stages 1–2 follow HMS-IDS [8]; Stage 3 and the explanation reuse the LLM prompt of the original extension [6].

## IV. Proposed Methodology

### A. Architecture

Fig. 1 shows the pipeline. A browser extension extracts details of the email senders, subject line and email body. The information is then sent to a local Flask service. Stage 1 determines the likelihood the email is a phishing email. If the email is determined to be legitimate in stage 1, it is passed to stage 2. From here, if it is found to still be legitimate but is unclear, it will be passed to stage 3 where the LLM can determine the outcome. The LLM will also create the explanation that will be shown to the user.

### B. Feature engineering

Subject and body are concatenated and represented by TF-IDF weights [13] over word unigrams and bigrams (minimum document frequency 3, at most 30,000 terms, sublinear term frequency). As in HMS-IDS, features are ranked by Information Gain (IG) [14]. For target $Y$ and the binary presence $X_t$ of term $t$,

$$IG(Y;X_t) = H(Y) - \sum_{v\in\{0,1\}} P(X_t{=}v)\, H(Y \mid X_t{=}v), \quad (1)$$

where $H$ is Shannon entropy. The number of retained terms $k \in \{1000, 3000, 5000\}$ is chosen on the validation split, giving $k = 5,000$. Nine hand-crafted indicators are appended, scaled to $[0, 1]$: number of URLs, URL presence, URLs with raw IP addresses, exclamation marks, upper-case ratio of body and subject, currency symbols, log body length, and occurrences of urgency or credential words (e.g., *verify*,

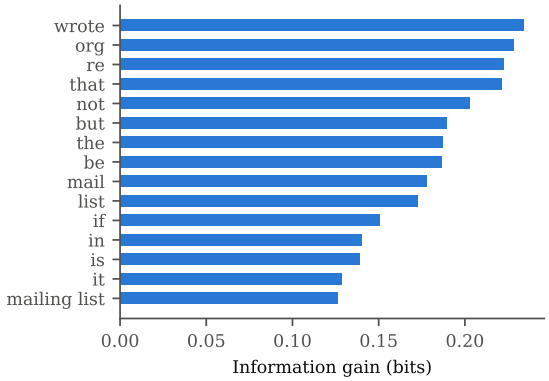



Fig. 2. Top-15 terms by Information Gain on the training split.

*suspend*, *password*). The final vector has 5,009 dimensions. Fig. 2 lists the highest-IG terms. They are not phishing cues but markers of the legitimate mailing-list discussions in CEAS (*wrote*, *re*, *mailing list*) and common function words. Most of the discriminative signal therefore comes from what legitimate email in this corpus looks like, which anticipates the generalisation problem in Section V-C.

### C. Stage 1: stacking ensemble

Following HMS-IDS, four base learners are trained: multinomial Naive Bayes, Decision Tree, KNN (cosine distance) and Logistic Regression. Each one's hyper-parameters (smoothing $\alpha$, maximum depth, number of neighbours, inverse regularisation $C$) are chosen on the validation split. The tuned learners are combined by stacking [15]. Out-of-fold class probabilities from 5-fold cross-validation train a meta-learner. As in HMS-IDS, the meta-learner is the algorithm of the best base learner on validation, which here is Logistic Regression. All models use scikit-learn [16].

### D. Stage 2: CL-$k$-means anomaly layer

Emails are embedded with truncated SVD (100 components) of the Stage 1 features and L2-normalised. Mini-batch $k$-means [17] partitions the training emails into $k$ clusters. Each cluster $C_j$ receives the majority label $M_j$ and the HMS-IDS confidence (purity)

$$\mathrm{conf}(C_j) = \frac{|\{x \in C_j : y(x) = M_j\}|}{|C_j|}. \tag{2}$$

An email that Stage 1 labels legitimate is re-labelled phishing if its nearest cluster has $M_j = 1$ and $\mathrm{conf}(C_j) \geq \tau$. Here we add the threshold $\tau$ to the HMS-IDS rule. The cluster count $k \in \{50, 100, 200, 400\}$ is chosen by the stand-alone validation F1 of CL-$k$-means ($k = 400$), and $\tau$ by the validation F1 of the full hybrid ($\tau = 1.0$).

### E. Stage 3: LLM escalation and explanation

Emails that remain legitimate but have a Stage 1 phishing probability $p \geq p_{low}$ (default 0.05) are sent to Gemini [18] with the zero-shot prompt of the original extension [6]. The prompt asks for a JSON answer with an `is_phishing` flag and a short explanation. A phishing verdict from the LLM overrides the legitimate label. Confident decisions never wait for an API call, and only uncertain emails leave the user's machine for classification.

TABLE II
TEST-SPLIT RESULTS (6,270 EMAILS). CV ACC. = 5-FOLD CV ACCURACY ON THE TRAINING SPLIT.

| Model | CV Acc. | Acc. (%) | Prec. | Rec. | F1 | FPR |
|---|---|---|---|---|---|---|
| Naive Bayes | 0.9182 | 91.87 | 0.9989 | 0.8473 | 0.9169 | 0.0010 |
| Decision Tree | 0.9774 | 98.28 | 0.9823 | 0.9852 | 0.9838 | 0.0200 |
| KNN | 0.8278 | 83.83 | 0.7663 | 0.9994 | 0.8675 | 0.3431 |
| Logistic Regression | 0.9939 | 99.51 | 0.9949 | 0.9958 | 0.9953 | 0.0058 |
| Stacking Ensemble | – | 99.54 | 0.9958 | 0.9955 | **0.9956** | 0.0047 |
| CL-k-means (alone) | – | 98.25 | 0.9837 | 0.9831 | 0.9834 | 0.0183 |
| **Hybrid (proposed)** | – | 99.51 | 0.9952 | 0.9955 | 0.9953 | 0.0054 |

### F. Browser-extension integration

Rather than requesting the open email via an event listener that listens for changes to the Gmail page, we request it directly from the content script when the popup opens. We have created a new endpoint at `/api/hybrid-scan`, which runs stages 1-3 and returns the verdict, the chance of phishing and the stage at which the verdict was decided. The original LLM only endpoint has been retained.

### G. Evaluation metrics

We report accuracy, precision, recall, F1, and the false-positive rate $\mathrm{FPR} = FP/(FP + TN)$, with phishing as the positive class. The FPR matters because a flagged legitimate email erodes user trust.

## V. Results and Discussion

### A. In-distribution performance

Table II and Fig. 3 give test-split results. Logistic Regression is the strongest single learner (99.51% accuracy), and stacking improves it slightly to 99.54% (F1 0.9956). Stacking misses 15 phishing emails and raises 14 false alarms out of 6,270. KNN performs the worst. It has almost perfect recall but very high FPR. This is likely due to a large portion of dense engineered features causing cosine similarity to be dominating in an overall sparse feature space. Whilst NB had the lowest performance in HMS-IDS, NB has very low FPR but the lowest recall.

With the validation-selected $\tau = 1.0$, the hybrid achieves 99.51% accuracy (F1 0.9953). This is essentially the same as Stage 1: the anomaly layer re-labels 2 emails as phishing, of which 0 actually are, while Stage 1 had missed 15. If stage 1 is already at close to its maximum, adding a second unsupervised level gives little improvement. This also fits with HMS-IDS where improvements are also due to unknown attack scenario. Fig. 4 shows the confusion matrices.

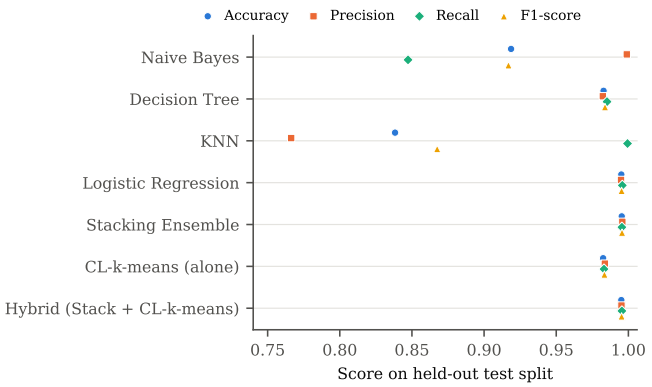



Fig. 3. Test metrics per model. The axis is zoomed because the dot plot has no baseline.

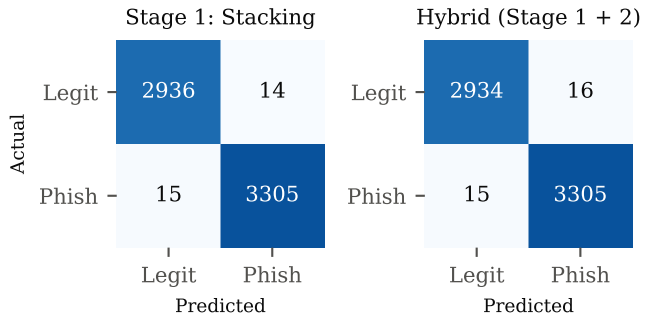



Fig. 4. Confusion matrices on the test split.

### B. Effect of the purity threshold $\tau$

Table III shows the security/usability trade-off controlled by $\tau$. Lowering $\tau$ to 0.8 raises in-distribution recall to 0.9988 (4 missed phishing emails) at the cost of an FPR of 1.15%. An organisation that prefers to over-flag could choose this operating point; a personal inbox would keep a strict $\tau$.

### C. Unseen-corpus (zero-day-style) evaluation

In order to simulate new phishing attacks, we train our model on a different corpus than it is tested on. We use the same hyper-parameters (Table IV, Fig. 5). The results change sharply. Trained on 27,254 CEAS emails and tested on all 2,858 Ling emails, the stacking ensemble's F1 drops from 0.9956 to 0.3647, with recall 0.2626. Its accuracy of 85.37% is barely above the 84.01% obtained by labelling every Ling email legitimate. The CL-$k$-means layer does not recover the loss (hybrid F1 0.2898): clusters learned on CEAS do not describe Ling's vocabulary. In the reverse direction, trained on only 2,001 Ling emails, the models also degrade (stacking F1 0.7920). Here the anomaly layer does recover a small number of missed phishing emails (hybrid recall 0.8755 vs. 0.8712), and CL-$k$-means on its own (F1 0.8367) outperforms the stacking ensemble. This suggests that cluster information can be better transferred than decision boundaries in situations where there is little training data.

TABLE III
HYBRID PERFORMANCE AS A FUNCTION OF THE PURITY THRESHOLD $\tau$ ($k = 400$)

| | In-distribution test | | | CEAS→Ling | | |
|---|---|---|---|---|---|---|
| $\tau$ | Rec. | FPR | F1 | Rec. | FPR | F1 |
| Stage 1 only | 0.9955 | 0.0047 | 0.9956 | 0.2626 | 0.0337 | 0.3647 |
| 0.6 | 0.9988 | 0.0207 | 0.9903 | 0.5252 | 0.4752 | 0.2612 |
| 0.7 | 0.9988 | 0.0176 | 0.9916 | 0.5230 | 0.4752 | 0.2602 |
| 0.8 | 0.9988 | 0.0115 | 0.9943 | 0.5098 | 0.4273 | 0.2716 |
| 0.9 | 0.9964 | 0.0088 | 0.9943 | 0.2954 | 0.1254 | 0.3024 |
| 0.95 | 0.9955 | 0.0071 | 0.9946 | 0.2691 | 0.1245 | 0.2799 |
| 0.99 | 0.9955 | 0.0054 | 0.9953 | 0.2691 | 0.1229 | 0.2811 |
| 1.0 | 0.9955 | 0.0054 | 0.9953 | 0.2670 | 0.1095 | 0.2898 |

TABLE IV
UNSEEN-CORPUS RESULTS: TRAIN ON ONE CORPUS, TEST ON THE OTHER

| | CEAS → Ling | | Ling → CEAS | |
|---|---|---|---|---|
| | Acc. | F1 | Acc. | F1 |
| Naive Bayes | 0.8464 | 0.0835 | 0.6592 | 0.7461 |
| Decision Tree | 0.7607 | 0.2484 | 0.6734 | 0.6627 |
| KNN | 0.1536 | 0.2654 | 0.4781 | 0.1301 |
| Logistic Regression | 0.8509 | 0.3983 | 0.8168 | 0.8279 |
| Stacking Ensemble | 0.8537 | 0.3647 | 0.7453 | 0.7920 |
| CL-k-means alone | 0.4878 | 0.2383 | 0.8153 | 0.8367 |
| **Hybrid (proposed)** | 0.7908 | 0.2898 | 0.7473 | 0.7941 |

This gap constitutes the main finding of our study. Learners based on bag-of-words will have learned the vocabulary used in the campaigns in the training set. This means that receiving a high in-distribution score of 99% tells us nothing about how they would perform on the next campaign. For text-based detectors, a near-perfect in-distribution score should therefore be complemented by a cross-corpus evaluation. This gap also provides motivation for Stage 3. As the LLM does not receive any corpus specific training, the errors it makes are not specific to the vocabulary of the training corpus. Section V-D tests whether this holds in practice.

### D. LLM stage

Stage 3 is implemented and runs in the extension (Section IV-F); however, we do not yet quantify this. We plan three experiments. 1) Compare Gemini to the ensemble in terms of zero shot accuracy on a balanced test set of emails. 2) Compare Gemini to the CEAS trained ensemble on the Ling emails (where Stages 1–2 fail). 3) Evaluate the three stage process on the test set, where only uncertain emails are escalated. Scripts and cached responses are available in the released code. Results will be presented in the updated version. Any claims that the LLM closes the generalisation gap remain hypotheses at this point.

### E. Run time

Fig. 6 reports training and inference time on a 2-core CPU. Stacking trains in 72.6 s and classifies the whole test split in 5.4 s. Most of that time is spent by its KNN member, which compares every email with the whole training set; the other

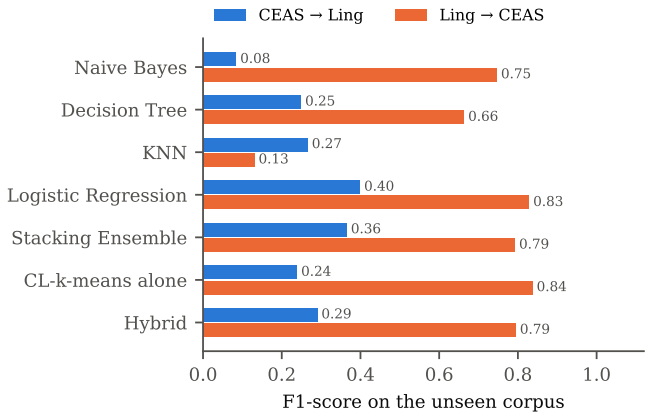



Fig. 5. F1 on the unseen corpus for both transfer directions.

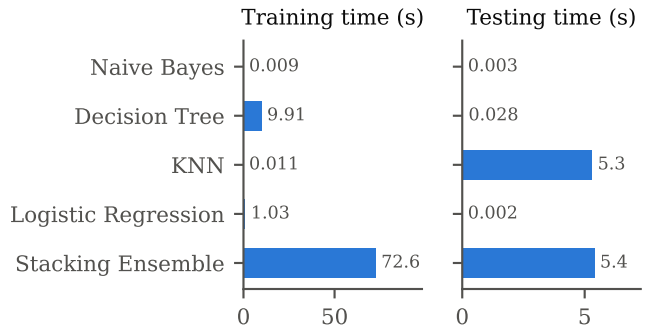



Fig. 6. Training and testing time per base learner (test split, 2-core CPU).

learners take a fraction of a second. This is one of the reasons Stage 3 is called only for uncertain emails.

## VI. Limitations and Future Work

Both corpora are quite dated (Ling-Spam was created in 2000, CEAS in 2008). Many of the samples in the 'phishing' class are actually spam samples. This means that the absolute numbers may not reflect current threats. Finally, exact duplicates are removed, meaning that near duplicates can still exist and potentially inflate performance on in-distribution data. The LLM stage inherits prompt-injection risk [7], and its verdicts depend on a model version that the provider can change or retire. Future work will add recent phishing corpora (e.g., the other datasets in [11]), use URL and email-header features, test lightweight local LLMs so that email content stays on the device, and update the clusters as new campaigns appear, as proposed by HMS-IDS.

## VII. Conclusion

We adapted the HMS-IDS two-tier architecture of a stacking ensemble followed by a cluster-labelling $k$-means anomaly layer from industrial-IoT intrusion detection to phishing email detection. We extended our algorithm with LLM escalation and implemented it in a Gmail browser extension. The ensemble achieved almost perfect performance in distribution (99.54% accuracy). Under an unseen-corpus protocol, however, its F1 drops to 0.3647. The unsupervised layer does not help close the gap. This shows that the most important thing when assessing a phishing detection system is to report the gap. To see how well the LLM layer helps close this, we can calculate it.

## Acknowledgment

We thank Dr. Kumar Saurabh, Bennett University, for his guidance throughout this work.

The browser extension and Gemini prompt build on the open-source CMPE209FinalProject by E. Pham, T. Bui and H. Chen [6]. The datasets were curated by Champa *et al.* [11].

## References

[1] I. Fette, N. Sadeh, and A. Tomasic, "Learning to detect phishing emails," in *Proc. 16th International Conference on World Wide Web (WWW)*, 2007, pp. 649–656.

[2] S. Abu-Nimeh, D. Nappa, X. Wang, and S. Nair, "A comparison of machine learning techniques for phishing detection," in *Proc. Anti-Phishing Working Groups 2nd Annual eCrime Researchers Summit*, 2007, pp. 60–69.

[3] T. Koide, N. Fukushi, H. Nakano, and D. Chiba, "ChatSpamDetector: Leveraging large language models for effective phishing email detection," arXiv preprint arXiv:2402.18093, 2024.

[4] F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park, "Devising and detecting phishing emails using large language models," *IEEE Access*, vol. 12, 2024.

[5] E. Pham, T. Bui, and H. Chen, "Demo: LLM-based browser extension for phishing detection," in *2025 Silicon Valley Cybersecurity Conference (SVCC)*, 2025, pp. 1–3.

[6] ——, "CMPE209FinalProject: Phishing extension using LLM," GitHub repository, https://github.com/Xainax/CMPE209FinalProject, 2025, san José State University, CMPE 209 course project.

[7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[8] K. Saurabh, V. Sharma, U. Singh, R. Khondoker, R. Vyas, and O. P. Vyas, "HMS-IDS: Threat intelligence integration for zero-day exploits and advanced persistent threats in IIoT," *Arabian Journal for Science and Engineering*, vol. 50, pp. 1307–1327, 2025.

[9] I. Androutsopoulos, J. Koutsias, K. V. Chandrinos, G. Paliouras, and C. D. Spyropoulos, "An evaluation of naive Bayesian anti-spam filtering," in *Proc. Workshop on Machine Learning in the New Information Age, 11th European Conference on Machine Learning*, 2000, pp. 9–17.

[10] A. I. Champa, M. F. Rabbi, and M. F. Zibran, "Curated datasets and feature analysis for phishing email detection with machine learning," in *Proc. 3rd IEEE International Conference on Computing and Machine Intelligence (ICMI)*, 2024, pp. 1–7.

[11] ——, "Phishing email curated datasets," Zenodo, 2023.

[12] M. M. Rahman, A. Rahman, and W. Sun, "Application of large language models for cybersecurity of power distribution grids: Challenges and opportunities," in *2025 57th North American Power Symposium (NAPS)*, 2025, pp. 1–6.

[13] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[14] Y. Yang and J. O. Pedersen, "A comparative study on feature selection in text categorization," in *Proc. 14th International Conference on Machine Learning (ICML)*, 1997, pp. 412–420.

[15] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] D. Sculley, "Web-scale k-means clustering," in *Proc. 19th International Conference on World Wide Web (WWW)*, 2010, pp. 1177–1178.

[18] Gemini Team, Google, “Gemini: A family of highly capable multimodal models,” arXiv preprint arXiv:2312.11805, 2023.